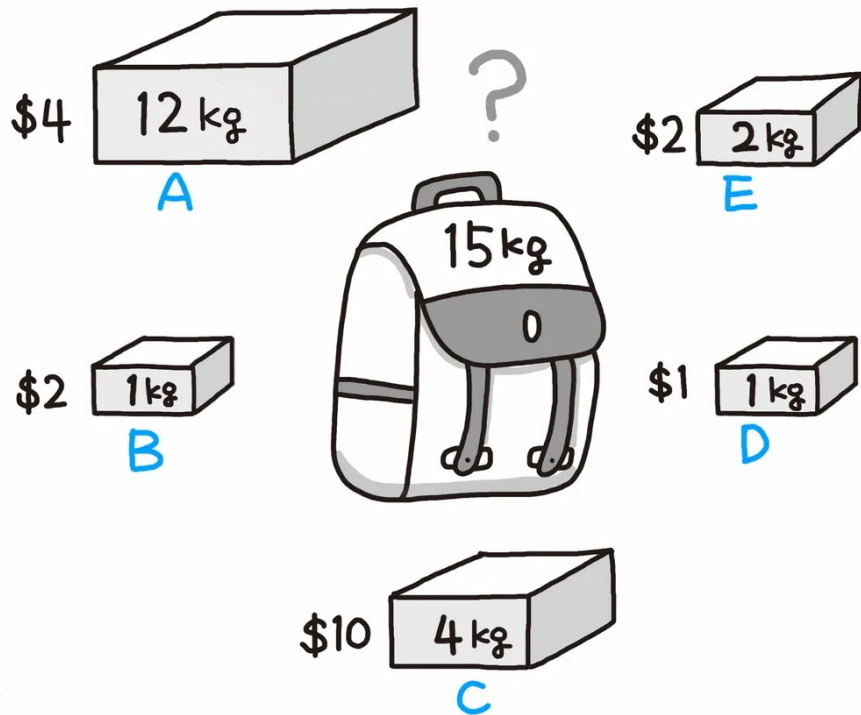


코딩 테스트 대비 알고리즘 스터디

7회차: DP 알고리즘 (4)

12865: 배낭 문제



문제

이 문제는 아주 평범한 배낭에 관한 문제이다.

한 달 후면 국가의 부름을 받게 되는 준서는 여행을 가려고 한다. 세상과의 단절을 슬퍼하며 최대한 즐기기 위한 여행이기 때문에, 가지고 다닐 배낭 또한 최대한 가치 있게 싸려고 한다.

준서가 여행에 필요하다고 생각하는 N 개의 물건이 있다. 각 물건은 무게 W 와 가치 V 를 가지는데, 해당 물건을 배낭에 넣어서 가면 준서가 V 만큼 즐길 수 있다. 아직 행군을 해본 적이 없는 준서는 최대 K 만큼의 무게만을 넣을 수 있는 배낭만 들고 다닐 수 있다. 준서가 최대한 즐거운 여행을 하기 위해 배낭에 넣을 수 있는 물건들의 가치의 최댓값을 알려 주자.

입출력 조건

입력

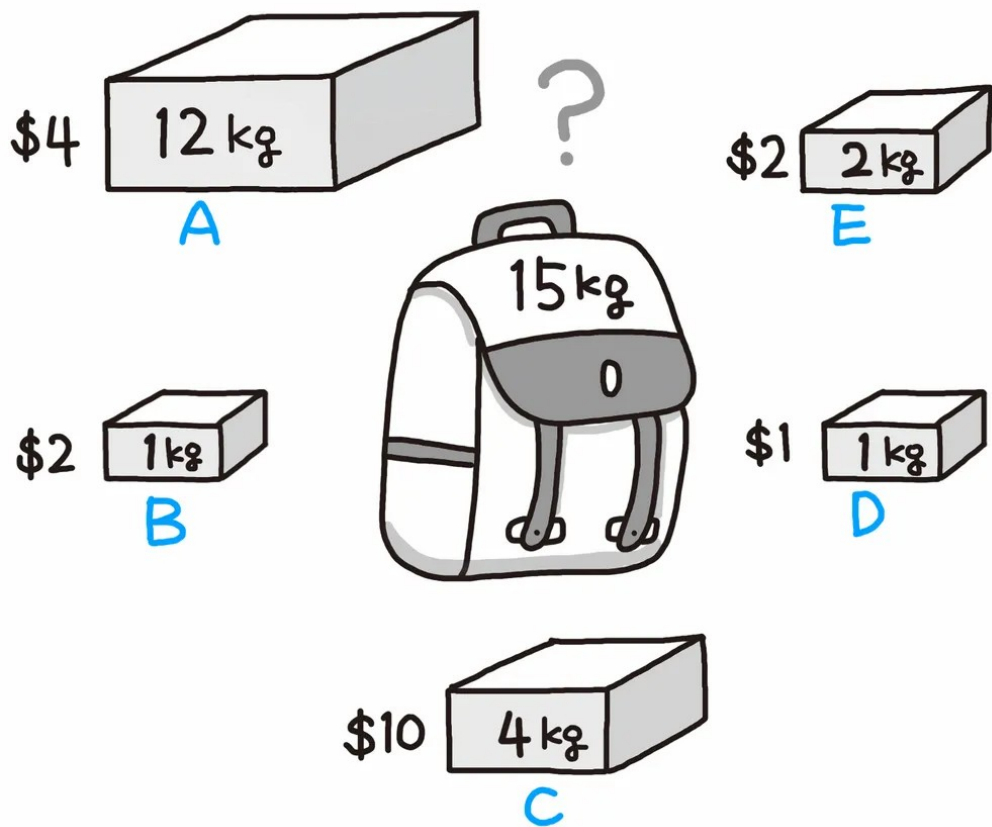
첫 줄에 물품의 수 N ($1 \leq N \leq 100$)과 준서가 버틸 수 있는 무게 K ($1 \leq K \leq 100,000$)가 주어진다. 두 번째 줄부터 N 개의 줄에 걸쳐 각 물건의 무게 W ($1 \leq W \leq 100,000$)와 해당 물건의 가치 V ($0 \leq V \leq 1,000$)가 주어진다.

입력으로 주어지는 모든 수는 정수이다.

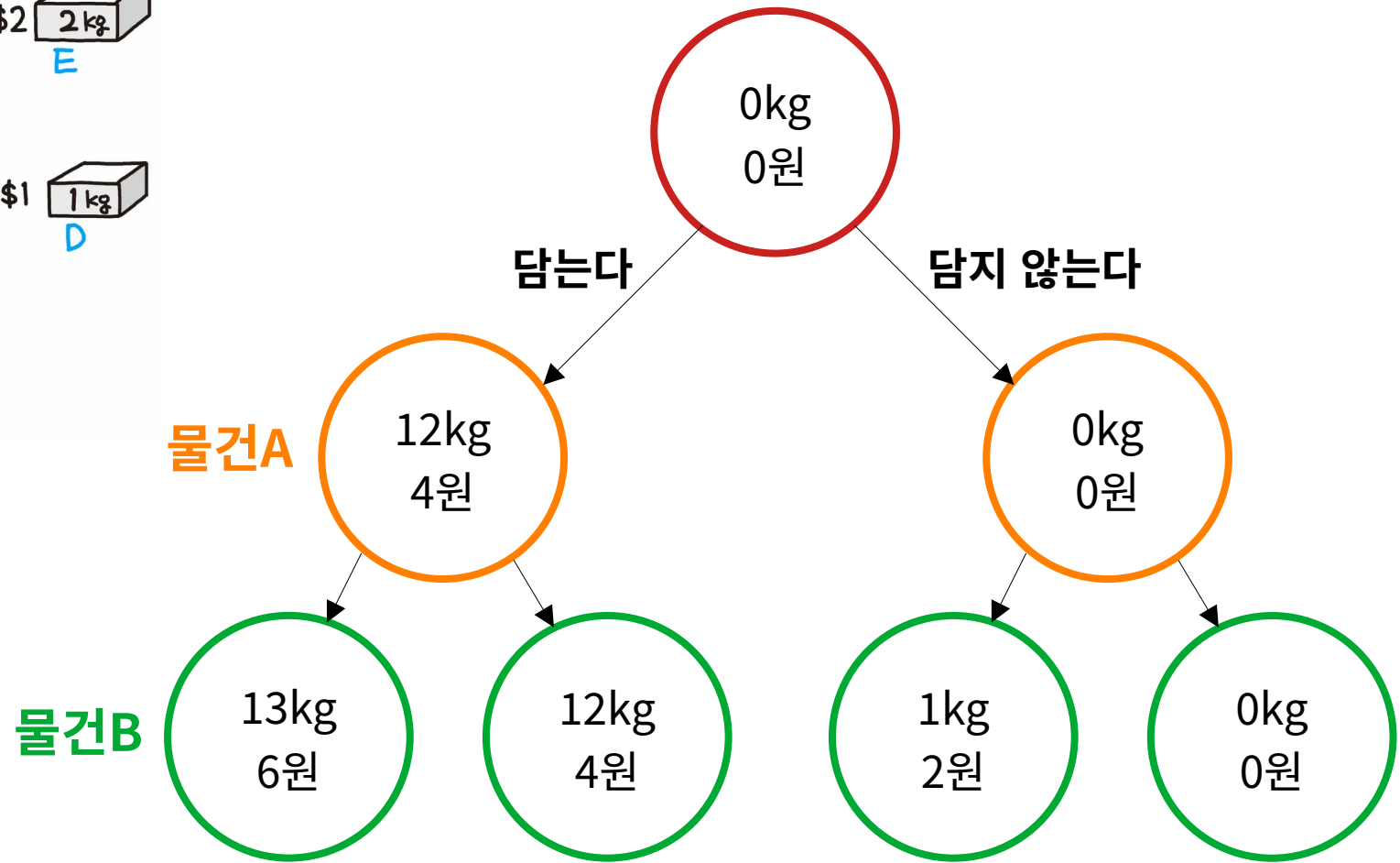
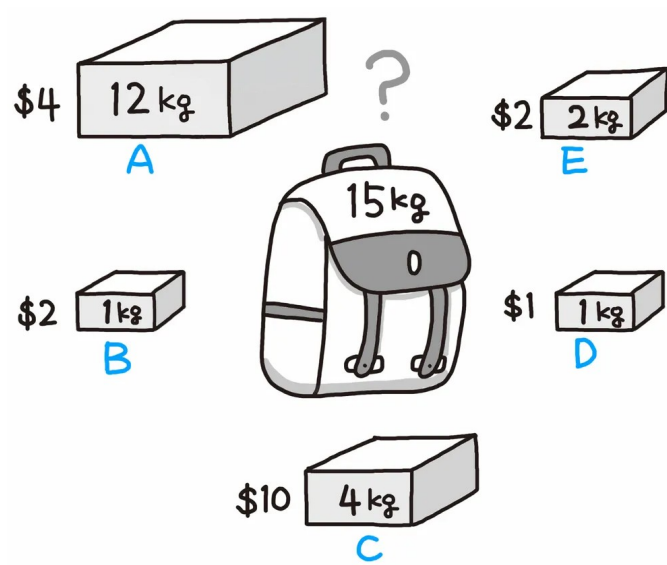
출력

한 줄에 배낭에 넣을 수 있는 물건들의 가치합의 최댓값을 출력한다.

문제 접근법 - Brute Force (DFS)



- 1) 15kg을 넘지 않는 조합 완전 탐색
- 2) 모든 조합에 대해 총 가치 계산
- 3) 가치 합 최대값



Brute-Force에서 중복되는 하위 문제

	A	B	C	D
무게	3	3	3	3
가치	1	2	3	4

 **중량 제한: 6kg**

1. A를 선택하고 B를 선택하지 않은 경우 → 가방에 남은 공간 3kg를 어떻게 채울 것인가?
2. A를 선택하지 않고 B를 선택한 경우 → 가방에 남은 공간 3kg를 어떻게 채울 것인가?

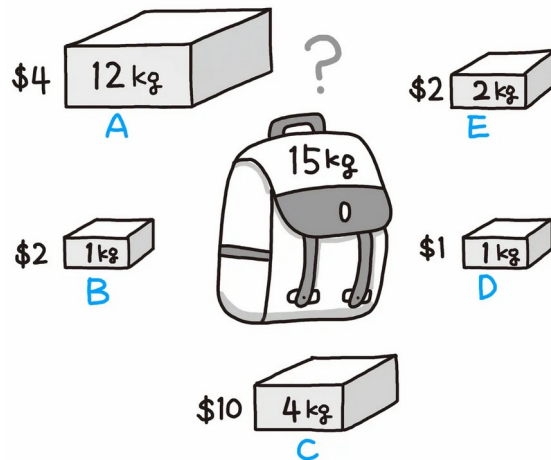
→ 동일한 하위 문제가 반복되고 있다

(i번째 물건까지 활용했을 때, 가방에 남은 공간 X를 채우는 최적의 방법)

문제 접근법 - DP

앞서 언급했던 중복되는 하위 문제의 2가지 핵심 변수:

- 1) 사용할 수 있는 물건의 종류 (DFS 기준 branch의 깊이)
- 2) 배낭의 무게 제한



**** 2차원 DP 배열로 풀이 ****

DP[i][j]

→ i번째 물건까지 활용해서 무게 제한 j인 배낭을 채우는 최대 가치

문제 접근법 - DP

$DP[i][j]$

→ i 번째 물건까지 활용해서 무게 제한 j 인 배낭을 채우는 최대 가치

이전 기록을 사용해서 $DP[i][j]$ 를 채우는 방법:

1. i 번째 물건이 무게 제한(j)을 초과하는 경우 (배낭에 담을 수 없다)

→ 직전 DP값 그대로 사용 ($DP[i-1][j]$)

2. i 번째 물건이 무게 제한을 초과하지 않는 경우

2-1) 물건을 담는 경우

→ 현재 물건을 제외한 DP값에 현재 값을 추가

→ $DP[i-1][j - \text{weight}[i]] + \text{value}[i]$

2-2) 물건을 담지 않는 경우

→ 직전 DP값 그대로 사용 ($DP[i-1][j]$)

백준 입력 예제

예제 입력 1 복사

```
4 7
6 13
4 8
3 6
5 12
```

물품 번호	1	2	3	4
무게	6	4	3	5
가치	13	8	6	12

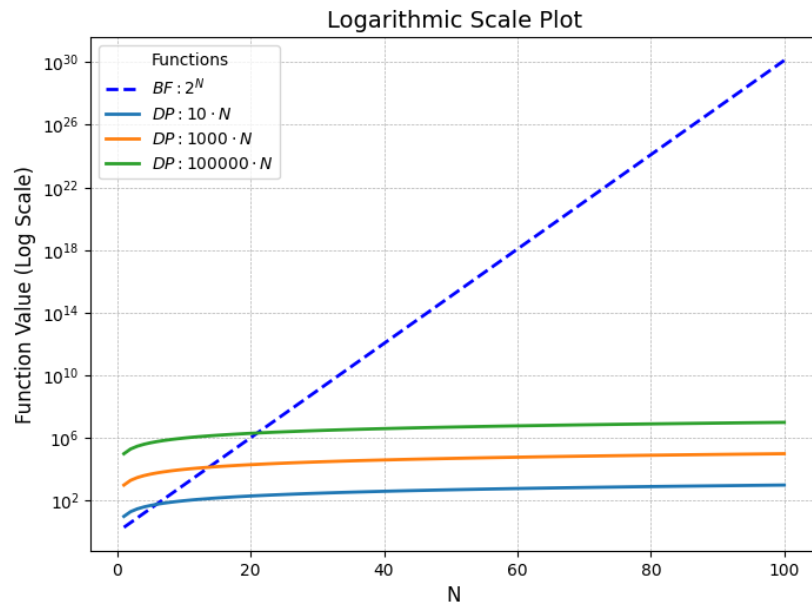
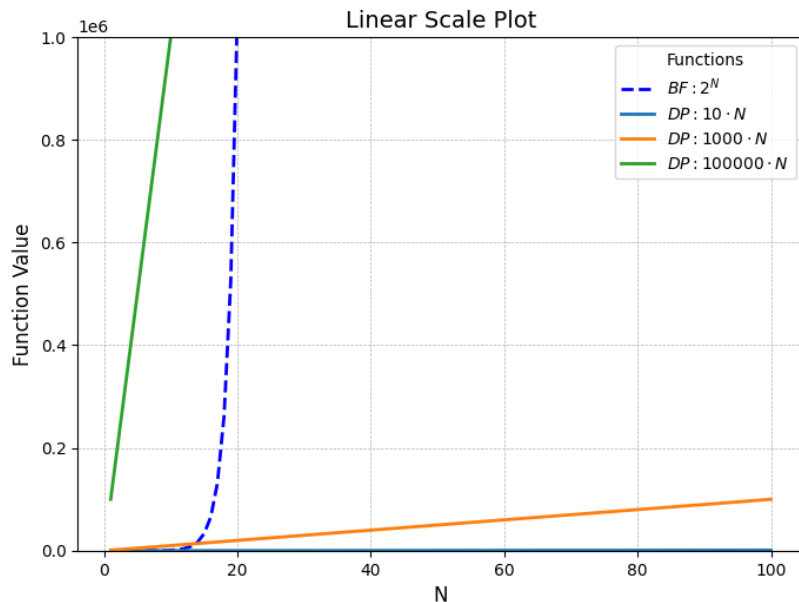
		무게 제한							
		0	1	2	3	4	5	6	7
물건 번호	0	0	0	0	0	0	0	0	0
	1	0	0	0	0	0	0	0+13=13 0	0+13=13 0
	2	0	0	0	0	0+8=8 0	0+8=8 0	0+8 13	0+8 13
	3	0	0	0	0+6	0+6 8	0+6 8	0+6 13	8+6=14 13
	4	0	0	0	6	8	0+12=12 8	0+12 13	0+12 14

Brute-Force와 DP의 시간복잡도 비교

Brute-Force: $O(n) = 2^n$

DP: $O(n * K)$ **K: 최대 용량**

Comparison of Linear and Logarithmic Scales



시간 복잡도에 따른 최대 N

현대 컴퓨터의 연산 능력 → 초당 약 10^8 회 → Python은 조금 느리므로 안전하게 10^7

시간 복잡도	안전한 N (1초 제한 기준)
$O(N^2)$	$N^2 \leq 10^7 \rightarrow N \leq 10^3 = \boxed{1000}$
$O(2^N)$	$2^N \leq 10^7 \rightarrow N \leq \boxed{23}$